Fondamentaux MVC, API et formulaires

Mastère ESI

Des produits aux voyages : modèle, API, validation et formulaires

Laravel 13 · Séquence 1

Objectifs

- Créer, lire et afficher des données avec Eloquent et Blade, en expliquant chaque fichier.
- Concevoir le modèle `Voyage` et l'exposer en API JSON.
- Rejeter une saisie invalide et expliquer la réponse.
- Créer et modifier un voyage par formulaire.

Prérequis

- Le projet Laravel 13 installé en séance.
- Une base de développement configurée et accessible.
- Les bases de PHP et de HTML.

Sommaire

Partie	Contenu
1	Modèle, migration et table des produits
2	Insérer et lire avec Eloquent dans Tinker
3	Route, contrôleur et affichage Blade
Vérifier	Points de contrôle et exercices de reprise
4	Concevoir le voyage et l'exposer en API
5	Valider les données avec une Form Request
6	Créer et modifier un voyage par formulaire

1. Construire le stockage des produits

Objectif : afficher des produits avec leur libellé, leur stock et leur prix, comme dans la démonstration. Le projet est déjà installé ; le CRUD complet viendra ensuite.

Lancez `php artisan ...` **dans le terminal, à la racine du projet** (fichier `artisan`). Les extraits Tinker se saisissent dans Tinker ; les autres vont dans les fichiers indiqués. Vérifiez l'existant avant de recréer des fichiers ou d'insérer des doublons.

Créer le modèle et sa migration

```
php artisan make:model Produit -m
```

`Produit` est le nom de la classe, au singulier ; `-m` demande aussi une migration. Deux fichiers sont créés :

Fichier	Rôle
`app/Models/Produit.php`	Le modèle PHP qui permet de manipuler les produits avec Eloquent.
`database/migrations/..._create_produits_table.php`	La description versionnée de la création de la table. Le préfixe du fichier est horodaté.

Un objet `Produit` représente une ligne ; la table `produits` conserve les lignes en base. **Eloquent relie les objets PHP aux données relationnelles.** Dans la configuration standard utilisée ici, il associe le modèle `Produit` à la table `produits`.

Définir les colonnes dans la migration

Dans la méthode `up()` de la migration générée, complétez le bloc `Schema::create` ainsi. Conservez les imports, la classe de migration et la méthode `down()` générés par Artisan.

```
Schema::create('produits', function (Blueprint $table) {
    $table->id();
    $table->string('libelle');
    $table->integer('stock');
    $table->double('prix');
    $table->timestamps();
});
```

`id()` crée la clé primaire ; `libelle` contient du texte, `stock` un entier, `prix` un flottant. `timestamps()` ajoute `created_at` et `updated_at`, dates de création et modification gérées par Eloquent.

> **Note**
>
> `double('prix')` reprend le choix de la démonstration. Pour un prix réel, préférez un stockage décimal adapté, par exemple `$table->decimal('prix', 10, 2)` (10 chiffres au total, dont 2 après la virgule), afin d'éviter les approximations du stockage flottant. Changer l'affichage du prix ne change pas son type de stockage.

Exécuter la migration

La connexion à la base est celle configurée pour le projet dans `.env`.

```
php artisan migrate
php artisan migrate:status
```

La première commande exécute les migrations en attente ; la seconde permet de vérifier que la migration des produits a été exécutée. **Créer le fichier de migration ne crée pas encore la table.**

> **Attention**
>
> Modifier le fichier d'une migration déjà exécutée ne modifie pas automatiquement la base. Dans un projet déjà utilisé, on fait évoluer le schéma avec une nouvelle migration. Ne réinitialisez pas toute la base pour résoudre une erreur : vous pourriez perdre les données.

2. Manipuler les données avec Eloquent

Autoriser les champs de l'insertion groupée

Dans `app/Models/Produit.php` :

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Produit extends Model
{
    protected $fillable = ['libelle', 'stock', 'prix'];
}
```

`$fillable` autorise ces champs lors d'une affectation groupée, comme l'appel à `create()` ci-dessous. **Ce n'est pas une validation des valeurs** : cette liste n'impose pas, par exemple, que le stock soit positif. La validation sera travaillée avec les formulaires. Cette syntaxe de la séance reste prise en charge dans Laravel 13, dont la documentation présente aussi l'attribut PHP `#[Fillable(...)]`.

Insérer un premier produit dans Tinker

Tinker permet d'exécuter du PHP dans le contexte de l'application, sans créer d'interface web. Dans le terminal :

```
php artisan tinker
```

Puis **dans Tinker**, saisissez :

```
use App\Models\Produit;

$produit = Produit::create([
    'libelle' => 'Clavier',
    'stock' => 12,
    'prix' => 49.90,
]);

$produit->id;
```

`create()` enregistre une ligne et renvoie l'objet créé, avec son identifiant. Chaque nouvel appel ajoute une ligne : il ne remplace pas automatiquement le produit précédent. La commande agit sur la base configurée du projet ; utilisez votre environnement de développement.

Lire et filtrer

Toujours dans Tinker :

```
Produit::all();
Produit::find($produit->id);
Produit::where('stock', '>', 5)->get();
```

Expression	Résultat
`all()`	Une collection contenant tous les produits.
`find($produit->id)`	Le produit portant cet identifiant, ou `null` s'il n'existe plus.
`where(...)->get()`	Une collection des produits dont le stock dépasse strictement 5.

`find(1)` suppose que la ligne 1 existe ; utilisez plutôt l'identifiant renvoyé à la création. `where()` prépare le filtre et `get()` exécute la lecture. `all()` suffit pour ce petit catalogue ; une liste volumineuse appellera une pagination.

Contrôle reproductible, après l'insertion : cette vérification relit la ligne depuis la base et doit afficher `true`. Elle ne crée aucune nouvelle ligne.

```
$relu = Produit::find($produit->id);
$relu !== null && $relu->libelle === 'Clavier'
    && (int) $relu->stock === 12;
```

Saisissez ensuite `exit` pour revenir au terminal avant de lancer une nouvelle commande Artisan.

3. Relier une URL aux produits affichés

Créer le contrôleur

```
php artisan make:controller ProduitController
```

Dans `app/Http/Controllers/ProduitController.php` :

```php
<?php

namespace App\Http\Controllers;

use App\Models\Produit;

class ProduitController extends Controller
{
    public function index()
    {
        $produits = Produit::all();

        return view('produits.index', ['produits' => $produits]);
    }
}
```

Le contrôleur demande les données au modèle puis les transmet à la vue. La clé `'produits'` du tableau devient la variable `$produits` accessible dans Blade. La méthode `index()` sert ici à afficher la liste.

Déclarer la route

Dans le fichier existant `routes/web.php`, ajoutez l'import du contrôleur et la route. L'import de `Route` est généralement déjà présent : ne le dupliquez pas.

```php
use App\Http\Controllers\ProduitController;
use Illuminate\Support\Facades\Route;

Route::get('/produits', [ProduitController::class, 'index']);
```

Une requête HTTP **GET** sur `/produits` appelle la méthode `index()` du contrôleur. Le nom de l'URL, celui de la méthode et celui de la vue sont reliés par ce code ; leur association n'est pas automatique.

Créer la vue Blade

Créez le dossier `resources/views/produits`, puis son fichier `index.blade.php`. Le nom `produits.index` utilisé dans `view()` désigne ce fichier : le point sépare le dossier du nom de la vue, sans écrire l'extension `.blade.php`.

```blade
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Produits</title>
</head>
<body>
    <h1>Liste des produits</h1>
    <table>
        <thead>
            <tr><th>Libellé</th><th>Stock</th><th>Prix</th></tr>
        </thead>
        <tbody>
            @foreach ($produits as $produit)
                <tr>
                    <td>{{ $produit->libelle }}</td>
                    <td>{{ $produit->stock }}</td>
                    <td>{{ $produit->prix }} €</td>
                </tr>
```

```
            @endforeach
        </tbody>
    </table>
</body>
</html>
```

`@foreach` parcourt la collection : `$produits` représente l'ensemble, `$produit` la ligne courante. `{{ ... }}` affiche une valeur en échappant les caractères HTML ; conservez cette syntaxe pour les données. Blade est exécuté côté serveur : le navigateur reçoit le HTML produit, pas la requête Eloquent ni les directives Blade.

Ouvrir la page et expliquer son fonctionnement

Dans le terminal :

```
php artisan route:list --path=produits
php artisan serve
```

Ouvrez l'URL du projet suivie de `/produits`, généralement `http://127.0.0.1:8000/produits`. Le tableau doit contenir « Clavier », `12` et le prix, qui peut apparaître comme `49.9` avant formatage monétaire.

Le trajet à raconter : GET `/produits` → route → `ProduitController::index()` → `Produit::all()` → vue `produits.index` → HTML renvoyé au navigateur.

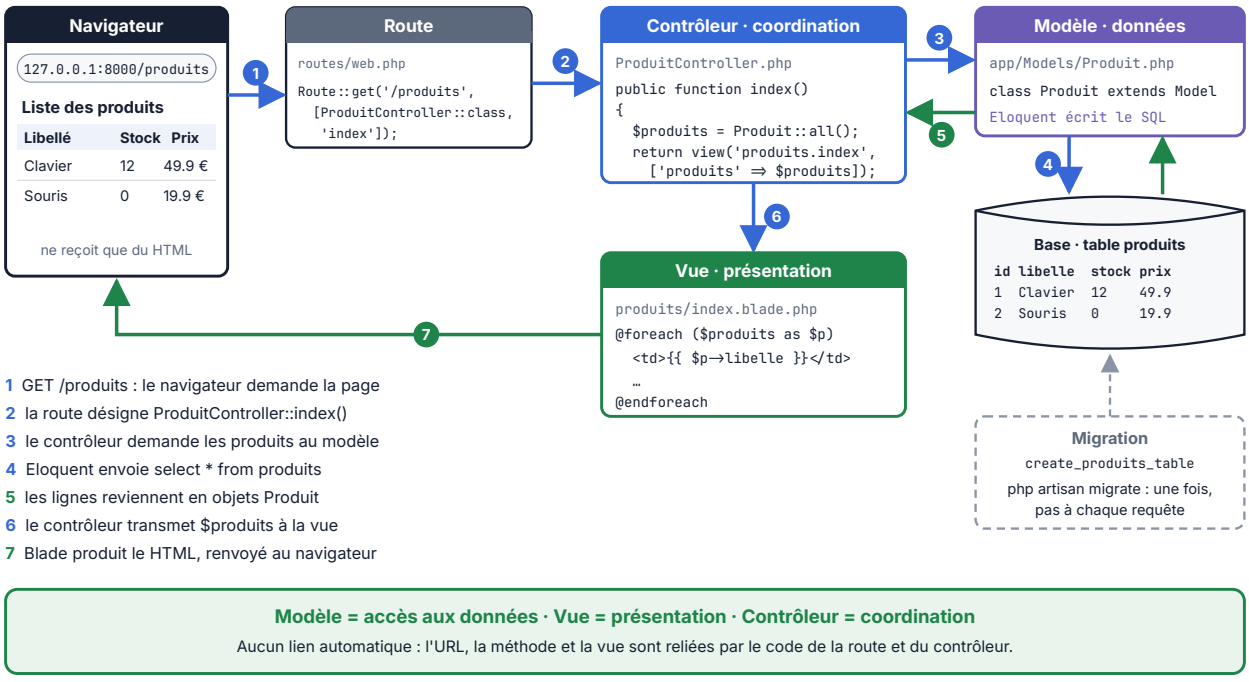


Fig. 1 - Trajet de GET /produits : route, contrôleur, modèle Eloquent, base, puis vue Blade et HTML renvoyé au navigateur ; la migration a préparé la table une seule fois.

MVC sépare les responsabilités : **modèle = accès aux données ; vue = présentation ; contrôleur = coordination de la demande**. La migration prépare la structure de stockage ; elle n'est pas exécutée à chaque consultation de la page.

Vérifier et reprendre la séance

Localiser une erreur

Symptôme	Premier contrôle
`Could not open input file: artisan`	Êtes-vous à la racine du projet ?
Connexion impossible ou table `produits` absente	Vérifiez la base configurée, puis `migrate:status`.
Erreur d'affectation groupée	Vérifiez `$fillable` dans le modèle et les noms des champs.
Route introuvable	Vérifiez l'URL et `route:list --path=produits`.
Vue introuvable ou variable non définie	Vérifiez le chemin du Blade et la clé `'produits'` transmise.
Tableau vide	Vérifiez `Produit::all()` dans Tinker : le Blade ne crée pas de données.

Trois exercices de reprise

Individuellement, sur le projet de développement. Ces exercices prolongent la démonstration. L'IA est autorisée pour expliquer une erreur ou relire votre tentative ; vous devez pouvoir expliquer les lignes conservées.

1. **Expliquer, sans modifier.** Sur votre feuille, associez la route, le contrôleur, le modèle, la migration et la vue à leur rôle. À quel endroit la lecture des produits est-elle déclenchée ?
2. **Ajouter et observer.** Avec Tinker, ajoutez une « Souris », stock `0`, prix `19.90`. Rechargez `/produits`. À produire : une page contenant le nouveau produit et l'explication de sa présence sans modification du Blade.
3. **Filtrer.** Modifiez uniquement la lecture dans le contrôleur pour n'afficher que les produits en stock strictement positif. À produire : « Clavier » reste visible et « Souris » disparaît de la page ; vérifiez dans Tinker que la souris existe toujours en base.

Sources

Support fondé sur l'échange de préparation de la séance. Syntaxe vérifiée le 9 septembre 2026 dans la documentation Laravel 13 via Context7 : migrations, Eloquent, routage et Blade. La compatibilité de `$fillable` est également vérifiée dans le code officiel de Laravel 13.

4. Concevoir le voyage et l'exposer en API

Objectif : passer des produits de démonstration au fil rouge « Voyages scolaires », avec un modèle conçu par l'équipe et une API JSON complète. Le travail se fait **à trois sur un seul dépôt** : une partie commune, puis une part autonome par personne.

Le dépôt commun

Le dépôt `voyages-scolaires` (Laravel 13, base SQLite) a été préparé par l'enseignant et vous y êtes invités comme collaborateurs. Chacun le clone et l'installe sur son poste :

```
git clone <url-du-depot> voyages-scolaires
cd voyages-scolaires
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate
```

`migrate` propose de créer le fichier `database/database.sqlite` : répondez oui. Le fichier `.env` et la base restent **hors du dépôt** (voir `.gitignore`) : chacun a les siens.

Concevoir le modèle avant de coder

Seul, sur papier, avant de lire la suite. Un parent consulte les voyages pour décider d'inscrire son enfant ; un enseignant les saisit. Listez les informations indispensables d'un voyage. Pour chacune, donnez son **type** et sa

contrainte métier (« obligatoire », « positif »…). À produire : un tableau de trois colonnes, confronté ensuite à ceux des autres.

Après mise en commun, l'équipe retient le schéma suivant. Les autres idées (date de retour, encadrants, participants inscrits) supposent des relations entre tables : elles viendront plus tard.

Attribut	Colonne	Contrainte métier
`titre`	`string`, 120 caractères	obligatoire
`destination`	`string`, 100 caractères	obligatoire
`depart`	`date`	obligatoire, postérieure à aujourd'hui
`prix`	`decimal(8, 2)`	obligatoire, strictement positif
`places`	`unsignedSmallInteger`	entier de 1 à 60

La troisième colonne n'est pas décorative : **elle deviendra les règles de validation** de la partie 5.

Programmer à trois : une personne au clavier

Un seul poste est projeté. La personne au clavier, le **pilote**, tape ; les deux autres, les **navigateurs**, lisent le support, vérifient chaque ligne et préviennent avant l'erreur. On change de pilote à chaque étape.

Étape 1, pilote 1 : modèle, migration et données de test.

```
php artisan make:model Voyage -mf
```

`-m` crée la migration, `-f` une *factory*, c'est-à-dire un générateur de voyages fictifs. Dans la migration :

```
Schema::create('voyages', function (Blueprint $table) {
    $table->id();
    $table->string('titre', 120);
    $table->string('destination', 100);
    $table->date('depart');
    $table->decimal('prix', 8, 2);
    $table->unsignedSmallInteger('places');
    $table->timestamps();
});
```

Dans `app/Models/Voyage.php`, déclarez les champs autorisés avec l'attribut `#[Fillable]` et ajoutez le *cast* :

```
use Illuminate\Database\Eloquent\Attributes\Fillable;

#[Fillable(['titre', 'destination', 'depart', 'prix', 'places'])]
class Voyage extends Model
{
    use HasFactory;

    protected function casts(): array
    {
        return ['depart' => 'date:Y-m-d'];
    }
}
```

`#[Fillable]` est un **attribut PHP 8** : une métadonnée attachée à la classe, que Laravel lit par réflexion. Il joue le même rôle que la propriété `protected $fillable = [...]` utilisée pour `Produit`, toujours acceptée par Laravel 13 et fréquente dans les projets existants.

Le *cast* transforme `depart` en objet date PHP (Carbon) à la lecture, et l'écrit au format `2027-03-15` dans le JSON. Dans `database/factories/VoyageFactory.php` :

```
public function definition(): array
{
    return [
        'titre' => fake()->sentence(3),
        'destination' => fake()->city(),
        'depart' => fake()->dateTimeBetween('+1 week', '+6 months'),
        'prix' => fake()->randomFloat(2, 50, 900),
        'places' => fake()->numberBetween(10, 60),
    ];
}
```

Dans la méthode `run()` de `database/seeders/DatabaseSeeder.php`, ajoutez `Voyage::factory(5)->create();` (avec `use App\Models\Voyage;` en haut du fichier), puis :

```
php artisan migrate
php artisan db:seed
```

Étape 2, pilote 2 : l'API en lecture.

```
php artisan install:api
php artisan make:controller Api/VoyageController --api --model=Voyage
```

`install:api` crée `routes/api.php` : toutes ses routes reçoivent le préfixe `/api`. `--api` génère un contrôleur sans `create()` ni `edit()`, qui affichent des formulaires inutiles à une API. Dans `routes/api.php` :

```
use App\Http\Controllers\Api\VoyageController;

Route::apiResource('voyages', VoyageController::class);
```

Cette ligne déclare cinq routes. Dans `app/Http/Controllers/Api/VoyageController.php` :

```
public function index()
{
    return Voyage::orderBy('depart')->get();
}

public function show(Voyage $voyage)
{
    return $voyage;
}
```

Laravel convertit en JSON le modèle ou la collection renvoyés. Le paramètre `Voyage $voyage` charge le voyage dont l'identifiant figure dans l'URL, ou répond **404** s'il n'existe pas.

```
php artisan route:list --path=api
curl -s http://127.0.0.1:8000/api/voyages
curl -s http://127.0.0.1:8000/api/voyages/1
```

> **Sous Windows**, utilisez `curl` dans Git Bash (dans PowerShell, `curl` désigne une autre commande), ou un client graphique comme Bruno.

Étape 3, pilote 3 : l'API en écriture, volontairement naïve.

```
public function store(Request $request)
{
    $voyage = Voyage::create($request->all());

    return response()->json($voyage, 201);
}

public function update(Request $request, Voyage $voyage)
{
    $voyage->update($request->all());

    return $voyage;
}

public function destroy(Voyage $voyage)
{
    $voyage->delete();

    return response()->noContent();
}
```

```
curl -s -X POST http://127.0.0.1:8000/api/voyages \
  -H "Accept: application/json" \
  -H "Content-Type: application/json" \
  -d '{"titre":"Rome antique","destination":"Rome",
       "depart":"2027-03-15","prix":420,"places":35}'
```

Code	Signification dans cette API
`200`	Lecture ou modification réussie, la ressource est dans la réponse.
`201`	Ressource créée.
`204`	Ressource supprimée, réponse vide.
`404`	Aucun voyage ne porte cet identifiant.
`422`	Données refusées par la validation (partie 5).

Étape 4, pilote 1 : la liste dans le navigateur. Reprenez la partie 3 : un contrôleur web `VoyageController` (sans `--api`) dont `index()` transmet `Voyage::all()` à la vue `voyages.index`, et la route nommée suivante dans `routes/web.php` :

```
Route::get('/voyages', [VoyageController::class, 'index'])
    ->name('voyages.index');
```

La vue affiche un tableau avec le titre, la destination, le départ et le prix. Le pilote enregistre et publie le travail commun sur `main` :

```
git add .
git commit -m "Modele Voyage, API CRUD et liste des voyages"
git push
```

Les deux autres récupèrent ce travail avec `git pull`, puis `php artisan migrate` et `php artisan db:seed` sur leur poste.

5. Valider les données avec une Form Request

La faille

Envoyez ces deux requêtes à l'API de l'étape 3 et observez les réponses :

```
curl -s -X POST http://127.0.0.1:8000/api/voyages \
  -H "Accept: application/json" \
  -H "Content-Type: application/json" \
  -d '{"titre":"Test","destination":"Lune",
       "depart":"2020-01-01","prix":-50,"places":0}'

curl -s -X POST http://127.0.0.1:8000/api/voyages \
  -H "Accept: application/json" \
  -H "Content-Type: application/json" \
  -d '{"titre":"","destination":"Lune",
       "depart":"2027-01-01","prix":10,"places":5}'
```

La première crée un voyage absurde : **201**. La seconde provoque une erreur **500** : la chaîne vide est convertie en `null` et la base refuse un titre nul. Aucune des deux réponses n'explique au client ce qu'il doit corriger.

Un attribut HTML `required` ne protège rien : `curl` ne passe pas par le formulaire. **La validation qui protège est celle du serveur** ; celle du navigateur n'apporte que du confort.

Écrire les règles une seule fois

```
php artisan make:request VoyageRequest
```

Dans `app/Http/Requests/VoyageRequest.php` :

```
public function authorize(): bool
{
    return true;
}

public function rules(): array
{
    return [
        'titre' => ['required', 'string', 'max:120'],
        'destination' => ['required', 'string', 'max:100'],
        'depart' => ['required', 'date', 'after:today'],
        'prix' => ['required', 'numeric', 'gt:0'],
        'places' => ['required', 'integer', 'between:1,60'],
    ];
}
```

Chaque règle traduit une ligne de la colonne « contrainte métier » de la partie 4. `authorize()` renvoie `true` pour l'instant : réserver la gestion aux enseignants sera l'objet de l'intervention 4. Dans le contrôleur d'API, remplacez `Request` par `VoyageRequest` et `$request->all()` par `$request->validated()` :

```
public function store(VoyageRequest $request)
{
    $voyage = Voyage::create($request->validated());

    return response()->json($voyage, 201);
}
```

Faites de même dans `update()`. `validated()` ne renvoie **que les champs validés** : un champ inattendu envoyé par le client est ignoré.

Deux clients, deux réponses

La Form Request est exécutée **avant** la méthode du contrôleur. Si une règle échoue, le contrôleur n'est jamais appelé et Laravel répond selon le client :

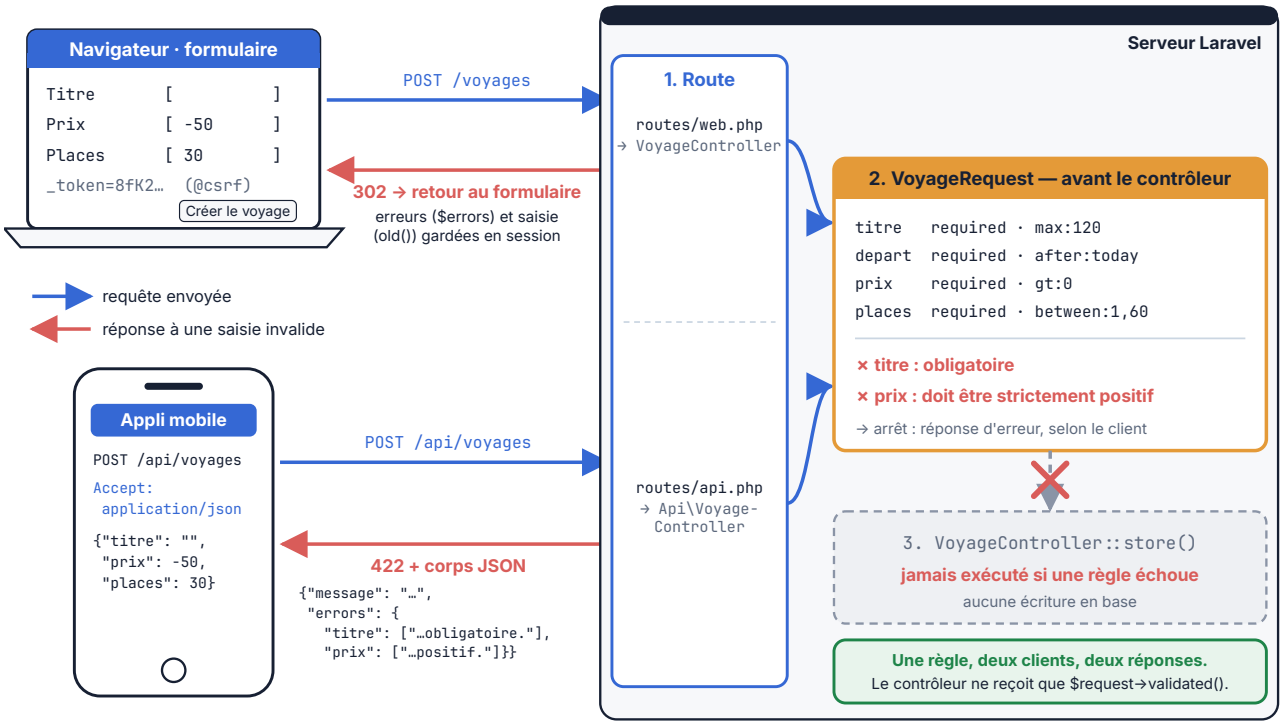


Fig. 2 - Trajet d'une saisie invalide : la Form Request l'arrête avant le contrôleur, puis répond par une redirection avec erreurs au navigateur, ou par un JSON 422 au client d'API.

Client	Réponse à une saisie invalide
Navigateur (formulaire)	Redirection `302` vers la page précédente ; erreurs et saisie conservées en session.
Client qui envoie `Accept: application/json`	`422` et un corps JSON `{"message": …, "errors": {…}}`.

> **Piège.** Sans l'en-tête `Accept: application/json`, `curl` est traité comme un navigateur : il reçoit une redirection `302`, pas la liste des erreurs.

La règle commune est publiée sur `main` par le pilote. Chacun crée ensuite **sa branche** à partir de cette version et y travaille seul :

```
git pull
git switch -c feature/<nom-de-la-part>
```

Les trois parts

Part	Branche	Fichiers principaux
API	`feature/api-validation`	`VoyageRequest.php`, `README.md`
Création	`feature/creation-voyage`	`routes/web.php`, `VoyageController.php`, `create.blade.php`, `index.blade.php`
Modification	`feature/modification-voyage`	`routes/web.php`, `VoyageController.php`, `edit.blade.php`, `index.blade.php`

Chacun commite sur sa branche et la publie avec `git push -u origin <branche>`. **Ne fusionnez pas dans `main`** : réunir les trois branches est le travail de l'après-midi.

Part « API » : une API qui explique ses refus

1. **Tester.** Remplissez ce tableau avec une requête `curl` par ligne :

Cas	Code obtenu	Message d'erreur
Voyage valide		
Titre vide		
Prix de −50 €		
61 places		
Départ hier		
Voyage valide avec un champ `"id": 999` en plus		
Titre vide, sans en-tête `Accept`		

2. **Traduire.** Les messages par défaut sont en anglais. Ajoutez à `VoyageRequest` une méthode `attributes()` (noms lisibles des champs) et une méthode `messages()` (phrases d'erreur), puis complétez-les pour toutes les règles :

```
public function attributes(): array
{
    return ['depart' => 'date de départ', 'places' => 'nombre de places'];
}

public function messages(): array
{
    return [
        'required' => 'Le champ :attribute est obligatoire.',
        'depart.after' => 'La date de départ doit être postérieure à aujourd\'hui.',
    ];
}
```

   Une clé limitée au nom de la règle (`required`) s'applique à tous les champs ; une clé `champ.règle` (`depart.after`) vise un seul champ. `:attribute` reçoit le nom lisible.

3. **Documenter.** Dans `README.md`, décrivez l'API pour un développeur de l'appli mobile : les cinq routes, un exemple de requête de création, les codes de réponse et le format d'une erreur `422`.

Critère de réussite : les sept cas sont testés et expliqués, chaque refus produit un message en français, le README permet d'utiliser l'API sans lire le code.

Extension

Écrivez un test de fonctionnalité (`php artisan make:test VoyageApiTest`) qui envoie un voyage de prix négatif avec `postJson('/api/voyages', [...])` et vérifie `assertStatus(422)` ; lancez-le avec `php artisan test`. Ce test sera exécuté automatiquement par la chaîne d'intégration continue du module DevOps.

6. Créer et modifier un voyage par formulaire

Les quatre mécanismes d'un formulaire Laravel

Élément	Rôle
`@csrf`	Ajoute un jeton secret propre à la session. Sans lui, Laravel refuse l'envoi (erreur **419**) : un autre site ne peut pas soumettre le formulaire à votre place.
`@method('PUT')`	Un formulaire HTML ne connaît que GET et POST : ce champ caché indique à Laravel la méthode réellement voulue (PUT, DELETE).
`@error('champ')`	Affiche, via `$message`, l'erreur de validation de ce champ s'il y en a une.
`old('champ')`	Restitue la saisie précédente après un refus, pour ne pas tout retaper.

Après un enregistrement réussi, le contrôleur redirige vers la liste avec un **message flash**, une donnée de session lue une seule fois :

```
return redirect()->route('voyages.index')
    ->with('succes', 'Voyage créé.');
```

Dans la vue, on l'affiche s'il existe :

```
@if (session('succes'))
    <p>{{ session('succes') }}</p>
@endif
```

Pour vérifier la validation **du serveur**, ajoutez temporairement l'attribut `novalidate` à la balise `<form>` : le navigateur cesse alors de bloquer lui-même les champs `type="number"` ou `type="date"` mal remplis.

Part « Création »

Routes, à ajouter sous la route de la liste dans `routes/web.php` :

```
Route::get('/voyages/create', [VoyageController::class, 'create'])
    ->name('voyages.create');
Route::post('/voyages', [VoyageController::class, 'store'])
    ->name('voyages.store');
```

Dans le contrôleur web :

```
public function create()
{
    return view('voyages.create');
}

public function store(VoyageRequest $request)
{
    Voyage::create($request->validated());
```

```
    return redirect()->route('voyages.index')
        ->with('succes', 'Voyage créé.');
}
```

Début de `resources/views/voyages/create.blade.php`, page HTML complète à construire autour :

```
<form method="POST" action="{{ route('voyages.store') }}">
    @csrf

    <label for="titre">Titre</label>
    <input id="titre" name="titre" value="{{ old('titre') }}">
    @error('titre')
        <p>{{ $message }}</p>
    @enderror

    <button type="submit">Créer le voyage</button>
</form>
```

1. **Compléter** le formulaire avec les quatre autres champs, sur le même modèle (`type="date"` pour le départ, `type="number"` pour le prix et les places, avec `step="0.01"` pour le prix).
2. **Relier** : dans la liste, un lien « Nouveau voyage » vers `route('voyages.create')` et l'affichage du message flash.
3. **Éprouver** : envoyez le formulaire vide, puis un prix négatif. À constater : les erreurs s'affichent sous les bons champs et les valeurs saisies sont conservées.
4. **Expliquer** : retirez `@csrf`, envoyez, notez le code obtenu, puis remettez la ligne. Pourquoi ce refus protège-t-il les utilisateurs ?

Critère de réussite : un voyage valide apparaît dans la liste avec le message de confirmation ; une saisie invalide revient sur le formulaire, avec ses erreurs et la saisie conservée.

Part « Modification »

Routes, à ajouter sous la route de la liste dans `routes/web.php` :

```
Route::get('/voyages/{voyage}/edit', [VoyageController::class, 'edit'])
    ->name('voyages.edit');
Route::put('/voyages/{voyage}', [VoyageController::class, 'update'])
    ->name('voyages.update');
Route::delete('/voyages/{voyage}', [VoyageController::class, 'destroy'])
    ->name('voyages.destroy');
```

Dans le contrôleur web :

```
public function edit(Voyage $voyage)
{
    return view('voyages.edit', ['voyage' => $voyage]);
}

public function update(VoyageRequest $request, Voyage $voyage)
{
    $voyage->update($request->validated());

    return redirect()->route('voyages.index')
        ->with('succes', 'Voyage modifié.');
}
```

Début de `resources/views/voyages/edit.blade.php` :

```
<form method="POST"
      action="{{ route('voyages.update', $voyage) }}">
    @csrf
    @method('PUT')

    <label for="titre">Titre</label>
    <input id="titre" name="titre"
           value="{{ old('titre', $voyage->titre) }}">
    @error('titre')
        <p>{{ $message }}</p>
    @enderror

    <button type="submit">Enregistrer</button>
</form>
```

`old('titre', $voyage->titre)` affiche la saisie refusée s'il y en a une, sinon la valeur enregistrée.

1. **Compléter** le formulaire avec les quatre autres champs. Attention au départ : un champ `type="date"` attend le format `2027-03-15`, alors que `$voyage->depart` est un objet date. Utilisez `$voyage->depart->format('Y-m-d')` comme valeur par défaut.
2. **Supprimer** : écrivez `destroy()` (redirection avec « Voyage supprimé. »), puis ajoutez à chaque ligne de la liste un lien « Modifier » et un petit formulaire de suppression contenant `@csrf`, `@method('DELETE')` et un bouton. Affichez le message flash dans la liste.
3. **Éprouver** : modifiez un prix en −10, puis en 15. À constater : le refus, puis la modification visible dans la liste.
4. **Raisonner** : avec Tinker, passez le départ d'un voyage à une date d'hier, puis essayez de corriger seulement son prix par le formulaire. Que se passe-t-il ? Quelle règle en est responsable ? Proposez une solution, sans forcément la coder.

Critère de réussite : un voyage se modifie et se supprime depuis la liste ; une modification invalide revient sur le formulaire prérempli avec ses erreurs.

Sources et sinks : où naissent les failles triviales

La validation contrôle ce qui **entre** ; elle ne suffit pas à rendre une donnée inoffensive. Un titre `<script>alert(1)</script>` respecte `max:120` : il est valide et sera enregistré.

- Une **source** est toute donnée contrôlée par le client : champ de formulaire, corps JSON, paramètre d'URL, en-tête, ou ligne de base qu'un client a remplie.
- Un **sink** est l'endroit où cette donnée est **interprétée** et non simplement affichée : une page HTML, une requête SQL, une URL de redirection, une commande système.
- Une faille apparaît quand une source atteint un sink **sans traitement adapté à son contexte**.

> **Valider en entrée, échapper en sortie.** La validation vérifie la règle métier ; l'échappement dépend de l'endroit où la donnée arrive. On ne « nettoie » pas un titre avec `strip_tags` à l'entrée : « Sortie » est un titre légitime.

Le JSON renvoyé par l'API avec un code `201` n'est pas un sink : servi en `application/json`, il n'est pas interprété comme du HTML par le navigateur. Le danger apparaît chez le client qui l'insère ensuite dans une page.

Sink	Faille	Écriture sûre
`{!! $voyage->titre !!}`	XSS stockée	`{{ $voyage->titre }}`, échappé par Blade
`element.innerHTML = voyage.titre`	XSS côté client	`element.textContent = voyage.titre`
`whereRaw("destination = '$d'")`	Injection SQL	`where('destination', $d)`, requête préparée
`redirect($request->input('retour'))`	Redirection ouverte	`redirect()->route('voyages.index')`
`#[Unguarded]` et `create($request->all())`	Affectation de masse	`#[Fillable]` et `validated()`

Exercice commun : chasser les sinks

À trois, après vos parts. Pour chaque extrait, nommez la **source**, le **sink** et la faille, puis proposez l'écriture sûre. Cherchez ensuite si l'un de ces sinks existe dans votre propre code.

1. ```
   <h2>{!! $voyage->titre !!}</h2>
   ```

2. ```
   $voyages = DB::select(
       "select * from voyages where destination = '" . $request->query('ville') . "'"
   );
   ```

3. ```
   const reponse = await fetch('/api/voyages');
   for (const v of await reponse.json()) {
       liste.innerHTML += `<li>${v.titre}</li>`;
   }
   ```

4. ```
   return redirect($request->input('retour', '/voyages'));
   ```

5. ```
   #[Unguarded]
   class Voyage extends Model {}

   // dans le contrôleur
   $voyage->update($request->all());
   ```

Critère de réussite : pour chaque extrait, la donnée dangereuse est suivie de la source au sink, et la correction agit au bon endroit.

Restitution de l'équipe

Chacun, à tour de rôle, sur son poste projeté :

1. montre une saisie invalide sur son interface et la réponse obtenue ;
2. raconte le trajet de cette requête, de l'envoi jusqu'à la réponse ;
3. choisit une ligne de son code et explique pourquoi elle est nécessaire ;
4. désigne un sink de sa part et montre ce qui le protège.

Les deux autres posent au moins une question. La séance est réussie quand chacun sait expliquer **pourquoi le contrôleur n'est jamais exécuté** lorsqu'une saisie est invalide.

Sources

Syntaxe vérifiée le 18 octobre 2026 dans la documentation Laravel 13 via Context7 : contrôleurs de ressource, routes d'API, Form Requests et réponse 422, simulation de méthode et ancienne saisie.
   ```

Corrigés

Mastère ESI

Des produits aux voyages : modèle, API, validation et formulaires

Laravel 13 · Séquence 1

1. Expliquer

La route associe l'URL à l'action ; le contrôleur orchestre la réponse ; le modèle permet l'accès aux lignes de `produits` ; la migration décrit le schéma ; la vue compose le HTML. La lecture est déclenchée par `Produit::all()` dans le contrôleur, et non par la boucle Blade.

2. Ajouter

Dans Tinker, après `use App\Models\Produit;` :

```
$souris = Produit::create([
    'libelle' => 'Souris',
    'stock' => 0,
    'prix' => 19.90,
]);
```

Au rechargement, le contrôleur relit les produits et la boucle affiche chaque ligne reçue. Le Blade n'a pas besoin de connaître à l'avance le nombre de produits.

3. Filtrer

Dans `index()`, remplacez la ligne de lecture par :

```
$produits = Produit::where('stock', '>', 0)->get();
```

La condition exclut de la lecture les produits de stock nul ; elle ne supprime aucune ligne. Dans la même session Tinker, `Produit::find($souris->id)` retrouve la souris. Si la session a été fermée, réimportez le modèle et recherchez-la avec `Produit::where('libelle', 'Souris')->get()`.

Partie 4 — Concevoir le voyage

`decimal(8, 2)` plutôt qu'un flottant : un prix se compte en centimes exacts. `places` en `unsignedSmallInteger` : jamais négatif, largement suffisant pour 60. La contrainte « postérieure à aujourd'hui » ne peut pas être une contrainte de colonne : elle dépend du jour de la saisie, d'où sa place dans la validation.

Partie 5 — Part « API »

Cas	Code	Explication
Voyage valide	`201`	Créé, le JSON contient l'`id` attribué.
Titre vide	`422`	`required` échoue.
Prix de −50 €	`422`	`gt:0` échoue.
61 places	`422`	`between:1,60` échoue.
Départ hier	`422`	`after:today` échoue.
Champ `"id": 999` en plus	`201`	`validated()` ignore `id` : la base attribue l'identifiant.
Titre vide sans `Accept`	`302`	Traité comme un navigateur : redirection, pas de JSON.

```
public function attributes(): array
{
    return [
        'titre' => 'titre',
        'destination' => 'destination',
        'depart' => 'date de départ',
        'prix' => 'prix',
        'places' => 'nombre de places',
    ];
}

public function messages(): array
{
    return [
        'required' => 'Le champ :attribute est obligatoire.',
        'max' => 'Le champ :attribute ne doit pas dépasser :max caractères.',
        'date' => 'Le champ :attribute doit être une date valide.',
        'depart.after' => 'La date de départ doit être postérieure à aujourd\'hui.',
        'numeric' => 'Le champ :attribute doit être un nombre.',
        'prix.gt' => 'Le prix doit être strictement positif.',
        'integer' => 'Le champ :attribute doit être un nombre entier.',
        'places.between' => 'Le nombre de places doit être compris entre 1 et 60.',
    ];
}
```

Extension, dans `tests/Feature/VoyageApiTest.php` :

```
public function test_un_prix_negatif_est_refuse(): void
{
    $this->postJson('/api/voyages', [
        'titre' => 'Test',
        'destination' => 'Rome',
        'depart' => now()->addMonth()->format('Y-m-d'),
        'prix' => -50,
        'places' => 30,
    ])->assertStatus(422)->assertJsonValidationErrors('prix');
}
```

Partie 6 — Part « Création »

Champ du départ, sur le modèle du titre :

```
<label for="depart">Départ</label>
<input id="depart" name="depart" type="date"
       value="{{ old('depart') }}">
@error('depart')
    <p>{{ $message }}</p>
@enderror
```

Sans `@csrf`, la réponse est **419** (*Page Expired*). Le jeton prouve que le formulaire vient bien de l'application : un site malveillant ouvert par un enseignant connecté ne peut pas lui faire créer un voyage à son insu (attaque CSRF).

Partie 6 — Part « Modification »

```
<input id="depart" name="depart" type="date"
       value="{{ old('depart', $voyage->depart->format('Y-m-d')) }}">
```

```
public function destroy(Voyage $voyage)
{
    $voyage->delete();

    return redirect()->route('voyages.index')
        ->with('succes', 'Voyage supprimé.');
}
```

```
<td>
    <a href="{{ route('voyages.edit', $voyage) }}">Modifier</a>
    <form method="POST"
          action="{{ route('voyages.destroy', $voyage) }}">
        @csrf
        @method('DELETE')
        <button type="submit">Supprimer</button>
    </form>
</td>
```

Raisonner : le formulaire renvoie le départ inchangé, et `after:today` le refuse. On ne peut plus corriger le prix d'un voyage passé. Solutions : deux Form Requests distinctes, `StoreVoyageRequest` avec `after:today` et `UpdateVoyageRequest` sans cette règle ; ou une seule classe qui n'applique `after:today` qu'à la création (`$this->isMethod('post')`). Le choix est métier : faut-il autoriser la modification d'un voyage déjà parti ?

Partie 6 — Chasser les sinks

1. **Source** : titre saisi, stocké en base. **Sink** : HTML de la page. **Faille** : XSS stockée, le script s'exécute chez chaque visiteur. **Sûr** : `{{ $voyage->titre }}`.
2. **Source** : paramètre `ville` de l'URL. **Sink** : requête SQL. **Faille** : injection SQL, `?ville=x' or '1'='1` renvoie tous les voyages. **Sûr** : requête préparée par Eloquent, `Voyage::where('destination', $request->query('ville'))->get()`.
3. **Source** : titre renvoyé par l'API. **Sink** : HTML via `innerHTML`. **Faille** : XSS côté client, alors que l'API est saine. **Sûr** : créer le `<li>` avec `document.createElement('li')` et remplir son `textContent`.
4. **Source** : champ `retour`. **Sink** : en-tête `Location`. **Faille** : redirection ouverte, `retour=https://site-piege.example` envoie l'utilisateur ailleurs depuis un lien de confiance. **Sûr** : `to_route('voyages.index')`, ou une liste de routes autorisées.
5. **Source** : corps de la requête. **Sink** : colonnes du modèle. **Faille** : affectation de masse, tout champ envoyé est écrit. **Sûr** : `#[Fillable([...])]` et `$voyage->update($request->validated())`.

L'extrait 3 montre que le danger se déplace avec la donnée : l'API renvoie un JSON correct, c'est le client qui crée le sink. L'extrait 5 est sans gravité sur `Voyage`, mais sur un modèle `User` avec une colonne `is_admin`, il permettrait de se déclarer administrateur.